

B-Tree vs B+ Tree — Complete Study Guide

Multilevel Indexing, Node Architectures, Search Paths & Range Query Traversal

1. WHY B-TREE AND B+ TREE?

Both data structures are used for multilevel indexing in database management systems (DBMS) to optimize disk I/O when searching large records.

INDEXING Record Components

-  **Key:** Search attribute used for lookup.
-  **Data Pointer:** Address pointing to the actual record in secondary storage.
-  **Block Pointer:** Address pointing to child tree nodes.

ANALOGY Book Index

Index Record: Topic Name (Key) + Page Number (Pointer)

↓

Data Record: Actual Text Content on Page

2. B-TREE STRUCTURE

Core Invariant

In a B-Tree, the structure of nodes remains **identical at every level** (Root, Internal, and Leaf).

B-Tree Node Layout (All Levels)

[Ptr₁ | Key₁ | DataPtr₁ | Ptr₂ | Key₂ | DataPtr₂ | Ptr₃]

At Leaf level, Block Pointers are set to `NULL`, but the field layout remains identical.

3. B+ TREE STRUCTURE

B+ Tree separates routing logic from record access by defining two distinct node formats:

1. Internal / Non-Leaf Node

Contains **Keys + Block Pointers** (No Data Pointers).

[BlockPtr | Key | BlockPtr]

Frees space to store more keys and child links per node.

2. Leaf Node

Contains **Keys + Data Pointers + Leaf Link**.

[Key | DataPtr] → [Key | DataPtr]

All actual data references are consolidated here.

4. NODE CAPACITY & SEARCH TRAVERSAL

Why B+ Tree Has More Children

Because internal nodes omit Data Pointers, more disk block space is available for keys and block pointers. This leads to a higher **fan-out ratio**, resulting in a broader tree with less overall depth.

Searching Paths

B-Tree: Search can terminate at Root, Internal, or Leaf levels.

B+ Tree: Search always traverses down to the Leaf level for record pointers.

5. KEY DUPLICATION & RANGE QUERIES IN B+ TREE

REDUNDANCY Key Duplication

In B+ Trees, internal keys are strictly navigational. A key used in an internal node is duplicated at the leaf level alongside its Data Pointer.

LINKEDLIST Linked Leaf Nodes

Leaf nodes are linked together via pointers:

Leaf₁ → Leaf₂ → Leaf₃ → Leaf₄

Range Query Efficiency Example

Query: Find all student roll numbers from 100 to 150.

In B+ Tree: Navigate to 100 at leaf level, then traverse the linked list directly: 100 → 101 → 102 → ... → 150 without returning to upper tree levels.

6. COMPREHENSIVE FEATURE COMPARISON

FEATURE	B-TREE	B+ TREE
Data Pointer Location	Root, Internal, and Leaf nodes	Leaf nodes only
Internal Node Structure	Key + Data Pointer + Block Pointer	Key + Block Pointer
Leaf Node Structure	Key + Data Pointer + Block Pointer (NULL)	Key + Data Pointer + Leaf Link Pointer
Node Uniformity	Identical structure at all levels	Internal and Leaf structures differ
Key Duplication	No redundant keys	Keys duplicated (Navigation + Leaf copy)
Leaf Nodes Linked?	❌ No	✅ Yes (Forms a linked list)
Range Queries	Inconvenient (Requires tree re-traversal)	Highly efficient (Sequential scanning)
Capacity / Shape	Lower fan-out / Greater depth	Higher fan-out / Broader (less depth)

7. EXAM TRAPS & COMMON MISCONCEPTIONS ⚠️

Trap 1: Internal Data Pointers

Does B+ Tree store data pointers in internal nodes? ❌ **NO**. All data pointers live at the leaf level.

Trap 2: B-Tree Leaf Links

Are B-Tree leaves linked? ❌ **NO**. Only B+ Tree connects leaf nodes sequentially.

8. MEMORY SHORTCUTS & 10-SECOND REVISION

🧠 B-TREE: DATA EVERYWHERE

- Root / Internal / Leaf contain Data Pointers
- Same node structure across all levels
- Leaves are NOT linked
- Search may finish at any level

🧠 B+ TREE: DATA AT LEAVES

- Data Pointers strictly at Leaf level
- Keys repeated for navigation
- Leaves linked sequentially
- Broad tree with optimal range queries